

Nonlinear Finite Element Method - Assignment 2024/25

Kartik Suresh Tandel

January 30, 2025

1 Introduction

This short technical documentation accompanies a Python program that simulates a one-dimensional bar with two segments (Rod 1 and Rod 2). As shown in Figure 1, the bar is clamped at each outer end, while a force $F(t)$ is applied at the interface. Figure 2 illustrates how $F(t)$ ramps linearly from 0 up to $\hat{F}$ over the total time t_{tot} .

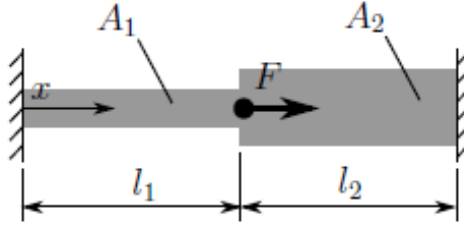


Figure 1: Schematic view of the bar, with cross-sections A_1 and A_2 , and lengths l_1 and l_2 . The force F is applied where the two segments meet.

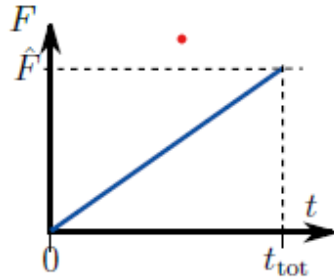


Figure 2: Loading history: the force $F(t)$ increases linearly from 0 at $t = 0$ to $\hat{F}$ at $t = t_{\text{tot}}$.

2 Implemented Methodology

2.1 Viscoplastic Constitutive Model

We assume small strains. The total strain is decomposed as

$$\varepsilon = \varepsilon_e + \varepsilon_p. \quad (1)$$

Here, ε_e is the elastic part and ε_p the (visco)plastic part

2.1.1 Governing Constitutive Equations

Our one-dimensional viscoplastic model is:

$$\sigma = E (\varepsilon - \varepsilon_p), \quad (2)$$

$$\dot{\varepsilon}_p = \eta \operatorname{sgn}(\sigma) \lambda, \quad (3)$$

where

$$\lambda = \left\langle \frac{|\sigma|}{\sigma_0} - 1 \right\rangle^m. \quad (4)$$

Above, E is Young's modulus, σ_0 is the yield stress, η is the viscosity parameter, and m is a strainratesensitivity exponent. The Macaulay bracket $\langle x \rangle = \max\{0, x\}$ ensures no plastic flow if $|\sigma| < \sigma_0$.

2.1.2 Backward-Euler Update

Over one time step Δt , the backward-Euler (fully implicit) update for plastic strain is

$$\varepsilon_{n+1}^p = \varepsilon_n^p + \Delta t \dot{\varepsilon}_{n+1}^p. \quad (5)$$

From (3),(4), at $n+1$ time we thus have

$$\varepsilon_{n+1}^p = \varepsilon_n^p + \Delta t \eta \operatorname{sgn}(\sigma_{n+1}) \left\langle \frac{|\sigma_{n+1}|}{\sigma_0} - 1 \right\rangle^m. \quad (6)$$

Meanwhile, the stress at step $n+1$ is given by (2), i.e.

$$\sigma_{n+1} = E (\varepsilon_{n+1} - \varepsilon_{n+1}^p). \quad (7)$$

2.1.3 Trial Stress ($\sigma_{n+1}^{\text{trial}}$) and Semi-Implicit Formula

- **Trial Stress:**

We define a “trial” stress (ignoring the new plastic increment) at time $n+1$ by

$$\sigma_{n+1}^{\text{trial}} = E (\varepsilon_{n+1} - \varepsilon_n^p). \quad (8)$$

Then compute the *trial* plastic multiplier:

$$\lambda_{n+1}^{\text{trial}} = \left\langle \frac{|\sigma_{n+1}^{\text{trial}}|}{\sigma_0} - 1 \right\rangle^m. \quad (9)$$

- **Elastic Case:**

If $\lambda_{n+1}^{\text{trial}} = 0$ (i.e. $|\sigma_{n+1}^{\text{trial}}| < \sigma_0$), then no plastic flow occurs:

$$\sigma_{n+1} = \sigma_{n+1}^{\text{trial}}, \quad \varepsilon_{n+1}^p = \varepsilon_n^p.$$

- **Plastic Case:**

Otherwise, we must solve

$$\sigma_{n+1} = E (\varepsilon_{n+1} - \varepsilon_{n+1}^p), \quad \varepsilon_{n+1}^p = \varepsilon_n^p + \Delta t \eta \operatorname{sgn}(\sigma_{n+1}) \left\langle \frac{|\sigma_{n+1}|}{\sigma_0} - 1 \right\rangle^m.$$

A common *semi-implicit* approach, we know that $\operatorname{sgn}(\sigma_{n+1}) = \operatorname{sgn}(\sigma_{n+1}^{\text{trial}})$. For the given case $m=1$, one obtains the closedform:

$$\sigma_{n+1} = \frac{\left[\sigma_{n+1}^{\text{trial}} + E \Delta t \eta \operatorname{sgn}(\sigma_{n+1}^{\text{trial}}) \right] \sigma_0}{\sigma_0 + E \Delta t \eta}. \quad (10)$$

Then we set

$$\varepsilon_{n+1}^p = \varepsilon_n^p + \Delta t \eta \operatorname{sgn}(\sigma_{n+1}) \left\langle \frac{|\sigma_{n+1}|}{\sigma_0} - 1 \right\rangle^m. \quad (11)$$

Then compute the plastic multiplier:

$$\lambda_{n+1} = \left\langle \frac{|\sigma_{n+1}|}{\sigma_0} - 1 \right\rangle^m. \quad (12)$$

2.1.4 Algorithmically Consistent Tangent Stiffness C_t

The material stiffness is Young's modulus when within the elastic region, when it reaches the yield point, we require the derivative of stress with respect to strain which further depends on the evolution of internal state variables.

$$C_t = \frac{\partial \sigma_{n+1}}{\partial \varepsilon_{n+1}}. \quad (13)$$

- **Elastic Regime:** If $|\sigma_{n+}^{\text{trial}}| < \sigma_0$, no plastic flow occurs and

$$C_t = E. \quad (14)$$

- **Plastic Regime (semi-implicit):** A standard result (see, e.g., lecture notes) is

$$C_t = \frac{\sigma_0 E}{\sigma_0 + E \eta \Delta t}. \quad (15)$$

Either of these is then used in the element stiffness calculation each Newton iteration.

2.2 Load Scaling in Nonlinear FEM

In nonlinear finite element analysis, solving for the equilibrium state under complex loading conditions often requires an incremental approach. Load scaling is a technique used to gradually apply the total load over several increments, facilitating convergence in nonlinear problems.

2.2.1 Load Scaling Technique

The total load is divided into smaller increments, and the problem is solved iteratively for each increment. This approach can be represented as:

$$\mathbf{F}_{\text{ext}}^{(i)} = \alpha_i \mathbf{F}_{\text{ext}} \quad (16)$$

where α_i is the load scaling factor at the i -th increment, and $\mathbf{F}_{\text{ext}}$ is the total external load. Typically, α_i varies as per the defined time steps, allowing the load to increase gradually from zero to its full magnitude.

2.3 Numerical Solution Techniques

Solving nonlinear FEM problems typically requires iterative methods due to the nonlinear nature of the equilibrium equations. The Newton-Raphson method is a common choice for its quadratic convergence properties. The technique involves linearizing the nonlinear equations about the current estimate of the solution and solving the resulting linear system to update the forecast. Convergence criteria based on the residual and displacement increments are used to determine when a sufficiently accurate solution has been found.

2.4 Newton-Raphson Method for Each Load Increment

The Newton-Raphson method is employed for each load increment to solve the nonlinear equilibrium equations. The iterative update for the displacement vector $\mathbf{u}$ at the k -th iteration within the i -th load increment is given by:

$$\mathbf{u}_{(i,k+1)} = \mathbf{u}_{(i,k)} + \Delta \mathbf{u}_{(i,k)} \quad (17)$$

where $\Delta \mathbf{u}_{(i,k)}$ is the solution to the linearized system:

$$\mathbf{K}_{(i,k)} \Delta \mathbf{u}_{(i,k)} = -\mathbf{R}_{(i,k)} \quad (18)$$

Here, $\mathbf{K}_{(i,k)}$ is the tangent stiffness matrix, and $\mathbf{R}_{(i,k)}$ is the residual vector.

2.5 Gauss Quadrature

Gauss quadrature is a numerical integration technique used to approximate the definite integrals of functions. It is particularly useful in the Finite Element Method (FEM) for integrating functions over elements, especially when dealing with nonlinear problems. In this assignment, we employ Gauss quadrature with two Gauss points to perform the numerical integration necessary for solving the nonlinear thermo-elastic-plastic problem in a composite sphere.

2.5.1 Concept

Gauss quadrature approximates the integral of a function $f(x)$ over an interval $[-1, 1]$ using a weighted sum of the function's values at specific points (Gauss points) within the interval:

$$\int_{-1}^1 f(x) dx \approx \sum_{i=1}^n w_i f(x_i) \quad (19)$$

where x_i are the Gauss points, w_i are the corresponding weights, and n is the number of Gauss points.

2.5.2 Two-Point Gauss Quadrature

For two-point Gauss quadrature, the Gauss points x_i and weights w_i are given by:

$$\begin{aligned} x_1 &= -\frac{1}{\sqrt{3}}, & w_1 &= 1 \\ x_2 &= \frac{1}{\sqrt{3}}, & w_2 &= 1 \end{aligned}$$

Thus, the integral can be approximated as:

$$\int_{-1}^1 f(x) dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right) \quad (20)$$

2.6 Element Routine

The element routine computes the element stiffness matrix and internal force vector using linear shape functions and numerical quadrature with two integration points.

$$\mathbf{K}_e = \sum_{i=1}^2 w(x_i) \mathbf{B}^T(x_i) \mathbf{C}_t(x_i) \mathbf{B}(x_i) J(x_i) \quad (21)$$

Explanation:

- $\mathbf{K}_e$: Element stiffness matrix
- $\sum_{i=1}^2$: Summation over the Gauss integration points (here 2 points)
- w_i : Weight associated with the i -th Gauss point
- $\mathbf{B}$: B matrix at the Gauss point
- $\mathbf{C}_t$: Tangent stiffness matrix at the Gauss point
- J : Determinant of the Jacobian matrix at the Gauss point

2.6.1 Global Stiffness Matrix (K)

$$\mathbf{K}_{\text{global}} = A_e^T \mathbf{K}_e A_e \quad (22)$$

Explanation:

- $\mathbf{K}_{\text{global}}$: Global stiffness matrix
- A_e : Assignment matrix that maps element quantities to the global system
- $\mathbf{K}_e$: Elemental stiffness matrix

2.6.2 Element Internal Force Vector ($F_{\text{int},e}$)

$$\mathbf{F}_{\text{int}}^e = w(x_i) \cdot \boldsymbol{\sigma} \cdot A \cdot \mathbf{B}(x_i) \cdot J(x_i) \quad (23)$$

Explanation:

- $\mathbf{F}_{\text{int}}^e$: Element internal force vector
- w : Weight at the Gauss point x_i
- $\boldsymbol{\sigma}$: Stress vector at the Gauss point
- A : Area of the element
- $\mathbf{B}$: Strain-displacement matrix
- J : Determinant of the Jacobian matrix at the Gauss point

2.6.3 Global Internal Force Vector (F_{int})

$$\mathbf{F}_{\text{int}} = A_e^T \mathbf{F}_{\text{int}}^e \quad (24)$$

Explanation:

- $\mathbf{F}_{\text{int}}$: Global internal force vector
- A_e : Assembly matrix that maps element quantities to the global system
- $\mathbf{F}_{\text{int}}^e$: Elemental internal force vector

3 Overview of the Program Structure

All routines are contained in a single Python file (e.g. `PVL.py`) and organized as follows more of it can be found in below Algorithm of the code: 1

- **material**: Implements the backward Euler update which are explained in the section:2.1 and returns σ_{n+1} , ε_{n+1}^p , and C_t .
- **elementStiffness**: Computes each element's stiffness and internal force by Gauss integration of the stress and element tangent stiffness from **material** routine.
- **main**: The top-level driver that sets geometry, material constants, load/time parameters, calls the solver, Assembles the global stiffness, calculates residual, applies boundary conditions, and solves the system via Newton iteration and plots results (displacement vs. time, stress vs. strain, etc.).

Algorithm 1 Finite Element Method for Viscoplastic Bar Simulation

```
1: Initialize: Define material properties, geometry, and time step.
2: Discretize: Define nodes, element lengths, and time increments.
3: Initialize storage arrays: Store stress, strain, displacement, and force data.
4: for each time step  $t$  do
5:     Apply external force at the interface node.
6:     for Newton-Raphson iterations do
7:         Compute the global stiffness matrix and internal force vector.
8:         for each element do
9:             Compute elemental stiffness and force via Gauss quadrature.
10:            Call material routine to update stress and plastic strain.
11:            Assemble global stiffness and force vectors.
12:        end for
13:        Apply boundary conditions (fixed displacement at both ends).
14:        Compute residual forces and solve for displacement increment.
15:        Update displacement and check for convergence.
16:        if convergence criteria met then
17:            Break from Newton-Raphson loop.
18:        end if
19:    end for
20:    Store results for stress, strain, and displacement.
21: end for
22: Post-processing: Generate plots for stress-strain, force-displacement, and displacement-
    time.
```

4 User Notes and Result Verification

4.1 Running the Code:

- **How to Run the Code:**

Use a command like `python PVL.py` in a terminal or any Python-supported IDE to run the script. The number of elements, time steps, and final load are predefined in the script and must be modified directly in the code if changes are needed.

- **Input Parameters:**

All required inputs are hardcoded in the Python code, stored in variables that match the chosen parameter variant (in our case, Variant 0).

- **Outputs:**

The script generates plots for:

- *Stress vs. Strain,*
- *External Force vs. Displacement,*
- *Displacement vs. Time.*

It also prints some numerical results to verify correctness, including:

- The computed displacement at an external load of $F_{\text{ext}} = 2100 \text{ N}$
- The load at which Bar 1 begins to yield,
- The load at which Bar 2 begins to yield.

4.2 Verification

Compare the yield load in each rod with the analytical estimate $\sigma_0 \cdot A$. We consider two bars in parallel (sharing the same node displacement u) and fixed at their outer ends. Let R_1 and R_2 be the internal forces in Bar 1 and Bar 2, respectively, with

$$R_1 + R_2 = F_{\text{ext}}. \quad (25)$$

If each bar remains linearly elastic, its elongation is

$$u_i = \frac{R_i L_i}{E A_i},$$

and because they share a common displacement at the load node,

$$u_1 = u_2 \implies \frac{R_1 L_1}{E A_1} = \frac{R_2 L_2}{E A_2} \implies \frac{R_1}{R_2} = \frac{A_1 L_2}{A_2 L_1}. \quad (26)$$

In your numerical example, one finds

$$R_1 = R_2,$$

so from (25) it follows

$$R_1 = R_2 = \frac{F_{\text{ext}}}{2}. \quad (27)$$

4.2.1 Elastic Displacement

Suppose each bar remains purely elastic at load=2100 , so

$$R_1 = R_2 = \frac{F_{\text{ext}}}{2} = 1050 \text{ N}.$$

In Bar 1 (with length $L = 40 \text{ mm}$, cross-sectional area $A = 8 \text{ mm}^2$, and Young's modulus $E = 80,000 \text{ MPa}$), the displacement is

$$u = \frac{R_1 L}{E A} = \frac{1050 \text{ N} \times 40 \text{ mm}}{80,000 \text{ MPa} \times 8 \text{ mm}^2} = 0.065625 \text{ mm}. \quad (28)$$

Thus, at $F_{\text{ext}} = 2100 \text{ N}$, the node displacement in Bar 1 from the FEM results would be $\approx 0.065625 \text{ mm}$ as shown in tabel 1 which matches the Analytical result. To check the consistent with an entirely elastic response up to load of 2560 N, which is the yielding load whose calculation can be seen below in section 4.2.2, we have plot which can be refereed from the figure 3. We can see that The stress vs strain, Displacement vs time and , Force vs Displacement curve increases linearly following the linear condition.

4.2.2 Analytical Yield Point Verification:

Yield Loads For each bar i , the yield condition is

$$\sigma_i = \frac{R_i}{A_i} = \sigma_0 \implies R_i = \sigma_0 A_i.$$

If R_i happens to be $\frac{F_{\text{ext}}}{2}$ (as in (27)), the external load at which *that* bar begins to yield is

$$F_{\text{ext, yield}, i} = 2 \sigma_0 A_i. \quad (29)$$

In our case for the set of parameteres variant-0,

$$\sigma_0 = 160 \text{ MPa}, \quad A_1 = 8 \text{ mm}^2, \quad A_2 = 16 \text{ mm}^2,$$

then

$$F_{\text{ext,yield},1} = 2 \times 160 \times 8 = 2560 \text{ N}, \quad F_{\text{ext,yield},2} = 2 \times 160 \times 16 = 5120 \text{ N}.$$

Thus Bar 1 reaches yield first, and Bar 2 only starts yielding at a higher load. The Bar 2 yields below its analytical external force, as shown above, as the Bar 1 plastically deforms which creates extra load on the Bar 2 and hence it plastifies at load of approximately 3846 N of external force which is found by the FEM code as shown in table 1. The FEM result and the analytical result match with the plastic initiation point with small errors as put in the table 1 for the first plastic initiation and its these value are also printed as an output when the code is run successfully. The Figure 4 shows the plastic initiation of Bar 1 and Bar 2 in stress vs strain curve plot and we can also look that, in Displacement vs time there is change in the slope two times. The first one shows the plastic initiation of the Bar 1 and latter one shows for both bars where at this point Bar 2 first time initiates plasticity.

Table 1: Comparison of FEM and Analytical Results (At 10 number element)

Type	FEM Results	Analytical Results	Error (%)
Displacement at load 2100 N	0.065625	0.065625	0
Plastic Initiation of Bar 1	2561	2560	0.01
Plastic Initiation of Bar 2	3846	-	-

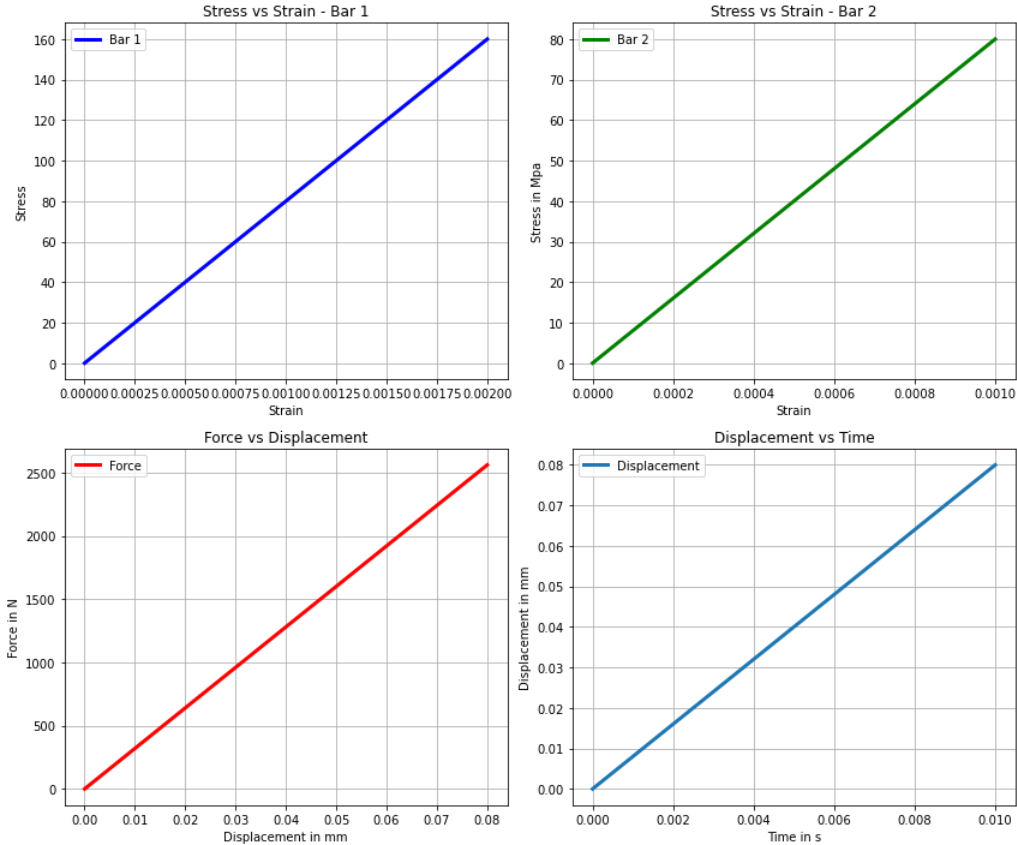


Figure 3: The Graph for the Load restricted to elastic condition.

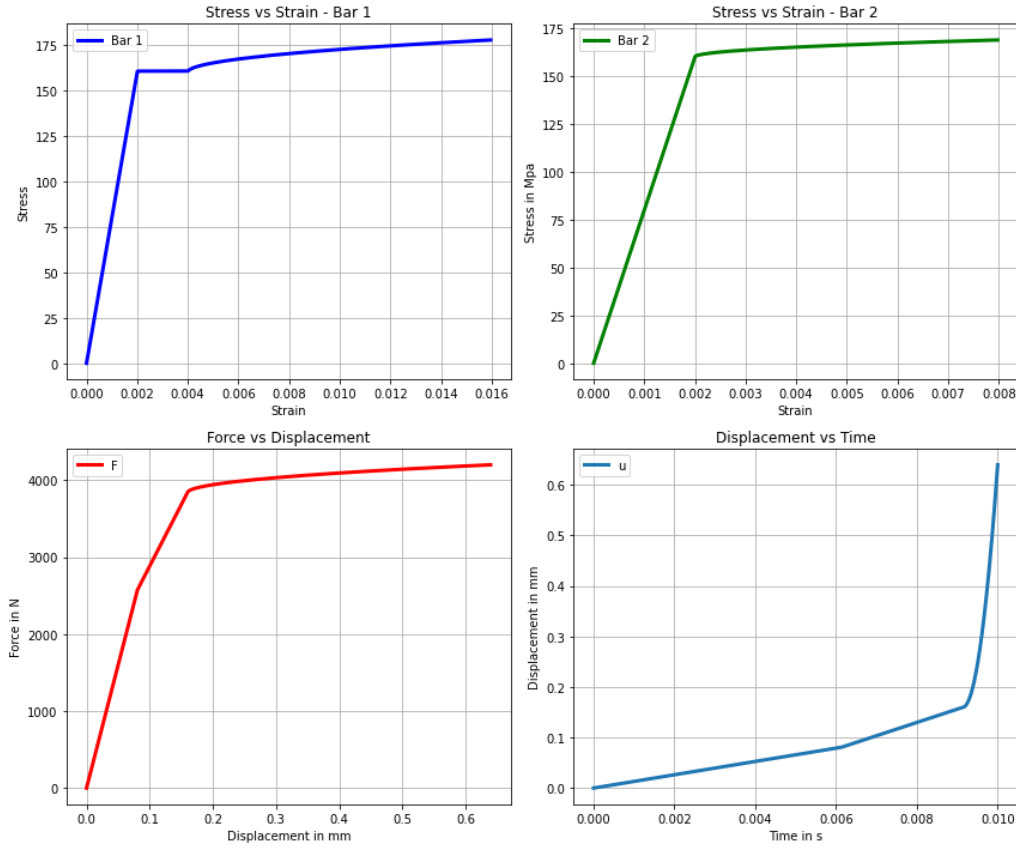


Figure 4: The Graph for the Load condition.

5 Convergence Study:

5.1 Introduction

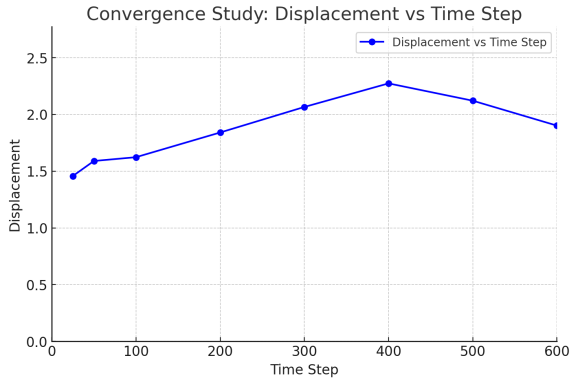
Convergence analysis is a crucial step in numerical simulations to ensure the accuracy and reliability of results. The convergence of the numerical solution depends on both the time step size and the element size. This study investigates how displacement results vary with changes in time step and element size.

5.2 Convergence with Time Step

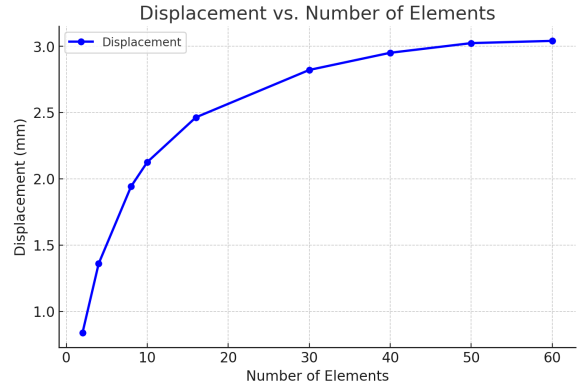
The first study examines the effect of varying time step sizes on the displacement values. The results are plotted in Figure 5a. The displacement initially increases as the time step decreases, indicating better resolution. However, after reaching an optimal time step, further refinement does not significantly improve the accuracy. Instead, numerical oscillations and accumulation of errors may lead to slight reductions in displacement values. This suggests that an optimal time step exists beyond which further refinement is unnecessary.

5.3 Convergence with Element Size

The second study evaluates the influence of element size on displacement accuracy, as shown in Figure 5b. Initially, with a lower number of elements, the displacement is underestimated due to poor spatial resolution. As the number of elements increases, the displacement values converge to a stable solution, demonstrating improved accuracy. After reaching approximately 50 elements, the change in displacement becomes negligible, indicating mesh convergence.



(a) Convergence with Time Step



(b) Convergence with Element Size

Figure 5: Comparison of Convergence with Time Step and Element Size.

5.4 Observations and Conclusions

Time Step Convergence: The displacement increases initially with a refined time step and stabilizes after an optimal point (400-500 time steps). Beyond this, excessive refinement does not yield better results and may introduce numerical instability.

Element Size Convergence: A small number of elements lead to inaccurate displacement values. Increasing the number of elements improves accuracy until a convergence point (50 elements), where further refinement does not significantly impact results.

Computational Efficiency Consideration: The study highlights the importance of selecting an optimal time step and mesh size. Excessively small time steps and mesh refinements increase computational cost without improving accuracy.